

Informe Triggers Módulo 7
Puente de Integración Desacoplada

Versión: 1.0

8 de mayo de 2026

Juan David Peña Roldán

Universidad Surcolombiana

Neiva, Huila

2026

TGR-M07-01:

trg_auditoria_peticiones_inmutable

Descripción	Impide cualquier operación UPDATE o DELETE sobre la tabla auditoria_peticiones, garantizando el modelo append-only exigido por RF-102. Al detectar un intento de UPDATE o DELETE sobre cualquier fila de auditoria_peticiones, lanza una excepción y cancela la operación.
Aplicado a	RF-102
Restricción origen RF	RF-102, Restricción 4.4 — <i>"No se permiten operaciones de tipo UPDATE, DELETE, PUT ni PATCH sobre los registros de auditoría. Esta restricción aplica para cualquier usuario o proceso, tanto a nivel de base de datos como de API."</i>
Justificación	La inmutabilidad debe ser garantizada a nivel de motor de base de datos, no solo a nivel de API. Un acceso directo a la BD (por DBA, script, migración mal ejecutada) podría violar la integridad si no existe esta protección en capa de datos.

```
CREATE OR REPLACE FUNCTION
modulo7.fn_auditoria_peticiones_inmutable()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        RAISE EXCEPTION
            '[TGR-M07-01] Violación de inmutabilidad: Los
registros de auditoría (RF-102) no pueden ser modificados. Evento:
%, Operación intentada: UPDATE.'
```

```

        OLD.evento;
    ELSIF TG_OP = 'DELETE' THEN
        RAISE EXCEPTION
            '[TGR-M07-01] Violación de inmutabilidad: Los
registros de auditoría (RF-102) no pueden ser eliminados. Evento:
%, Operación intentada: DELETE.',
            OLD.evento;
    END IF;
    RETURN NULL;
END;
$$;

CREATE OR REPLACE TRIGGER trg_auditoria_peticiones_inmutable
BEFORE UPDATE OR DELETE ON modulo7.auditoria_peticiones
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_auditoria_peticiones_inmutable();

COMMENT ON TRIGGER trg_auditoria_peticiones_inmutable ON
modulo7.auditoria_peticiones
IS 'TGR-M07-01 | RF-102 Restricción 4.4 | Garantiza inmutabilidad
absoluta (append-only) de todos los registros de auditoría del
módulo M07.';

```

TGR-M07-02:

trg_auditoria_peticiones_has h_integridad

Descripción	Al insertar un nuevo registro de auditoría, verifica que el campo evento_sha256 no llegue vacío. La BD no calcula el hash (eso es responsabilidad del backend), pero sí valida que el campo obligatorio esté presente y tenga el formato esperado de 64 caracteres hexadecimales. En cada INSERT sobre auditoria_peticiones, valida que evento_sha256 tenga exactamente 64
-------------	---

	caracteres y sea una cadena hexadecimal válida. Si no cumple, lanza excepción.
Aplicado a	RF-102
Restricción origen RF	RF-102, Restricción 4.6 — <i>"Cada registro debe incluir un campo hash_integridad (SHA-256) calculado sobre todos los campos del registro excepto el propio campo de hash, en el momento de su creación."</i> RF-102, Restricción 4.7 — <i>"Registro completo obligatorio: no se permite omitir campos obligatorios ni registrar eventos incompletos."</i>
Justificación	Aunque el backend calcula el hash, la BD debe actuar como última línea de defensa rechazando inserciones que lleguen con hash vacío, nulo o malformado, ya que un registro sin hash viola directamente la garantía de integridad de RF-102.

```

CREATE OR REPLACE FUNCTION modulo7.fn_auditoria_hash_obligatorio()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.evento_sha256 IS NULL OR TRIM(NEW.evento_sha256) = '' THEN
        RAISE EXCEPTION
            '[TGR-M07-02] Violación RF-102: El campo evento_sha256 es
obligatorio y no puede ser nulo o vacío. Evento UUID: %.',
            NEW.evento;
    END IF;

    IF LENGTH(NEW.evento_sha256) <> 64 THEN
        RAISE EXCEPTION
            '[TGR-M07-02] Violación RF-102: El campo evento_sha256 debe
tener exactamente 64 caracteres (SHA-256 hex). Longitud recibida: %.
Evento UUID: %.',
            LENGTH(NEW.evento_sha256),
            NEW.evento;
    END IF;

    IF NEW.evento_sha256 !~ '^[a-f0-9]{64}$' THEN
        RAISE EXCEPTION
            '[TGR-M07-02] Violación RF-102: El campo evento_sha256
contiene caracteres inválidos. Solo se permiten caracteres hexadecimales

```

```

en minúscula. Evento UUID: %.',
    NEW.evento;
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_auditoria_hash_obligatorio
BEFORE INSERT ON modulo7.auditoria_peticiones
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_auditoria_hash_obligatorio();

COMMENT ON TRIGGER trg_auditoria_hash_obligatorio ON
modulo7.auditoria_peticiones
IS 'TGR-M07-02 | RF-102 Restricción 4.6 y 4.7 | Valida que el hash
SHA-256 de integridad esté presente y sea formalmente válido en cada
registro de auditoría insertado.';

```

TGR-M07-03:

trg_versiones_contrato_transicion_estado

Descripción	Controla que las transiciones de estado en versiones_contrato_aaef sean estrictamente unidireccionales: PENDIENTE → ACTIVO → DEPRECADO → INACTIVO. Bloquea cualquier retroceso, salto de estado o reactivación desde INACTIVO. En cada UPDATE de la columna estado en versiones_contrato_aaef, valida que la transición sea válida según el orden definido. Si es inválida, lanza excepción y revierte.
Aplicado a	RF-98
Restricción origen RF	RF-98, Restricción 5.1 — <i>"Los estados del</i>

	<i>contrato solo pueden avanzar en el siguiente orden: PENDIENTE → ACTIVO → DEPRECADO → INACTIVO. No se permite regresar a estados anteriores, reactivar contratos INACTIVOS ni saltar estados intermedios." RF-98, Restricción 5.7 — "Toda transición de estado se ejecuta como operación atómica."</i>
Justificación	La secuencia de estados es una invariante de integridad estructural del contrato. Si un proceso del backend falla a mitad de una transición, la BD debe garantizar que no queden estados inválidos. Esta regla debe estar en el motor de datos.

```

CREATE OR REPLACE FUNCTION
modulo7.fn_versiones_contrato_transicion_estado()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_orden_anterior INT;
    v_orden_nuevo    INT;
BEGIN
    -- Mapeo de orden jerárquico de estados
    v_orden_anterior := CASE OLD.estado::TEXT
        WHEN 'PENDIENTE' THEN 1
        WHEN 'ACTIVO'    THEN 2
        WHEN 'DEPRECADO' THEN 3
        WHEN 'INACTIVO'  THEN 4
        ELSE 0
    END;

    v_orden_nuevo := CASE NEW.estado::TEXT
        WHEN 'PENDIENTE' THEN 1
        WHEN 'ACTIVO'    THEN 2
        WHEN 'DEPRECADO' THEN 3
        WHEN 'INACTIVO'  THEN 4
        ELSE 0
    END;

    -- Sin cambio de estado: permitir
    IF OLD.estado = NEW.estado THEN
        RETURN NEW;
    END IF;

```

```

-- Bloquear reactivación desde INACTIVO
IF OLD.estado::TEXT = 'INACTIVO' THEN
    RAISE EXCEPTION
        '[TGR-M07-03] Violación RF-98 Restricción 5.1: No se puede
reactivar un contrato en estado INACTIVO. Versión: %, Estado actual:
INACTIVO, Estado intentado: %.',
        OLD.contrato_version, NEW.estado;
END IF;

-- Bloquear retroceso de estado
IF v_orden_nuevo < v_orden_anterior THEN
    RAISE EXCEPTION
        '[TGR-M07-03] Violación RF-98 Restricción 5.1: Retroceso de
estado no permitido. Versión: %, Transición inválida: % → %.',
        OLD.contrato_version, OLD.estado, NEW.estado;
END IF;

-- Bloquear salto de estado (solo se permite avanzar un paso a la
vez)
IF v_orden_nuevo > v_orden_anterior + 1 THEN
    RAISE EXCEPTION
        '[TGR-M07-03] Violación RF-98 Restricción 5.1: Salto de
estado no permitido. Versión: %, Transición inválida: % → %. Solo se
permite avanzar un estado a la vez.',
        OLD.contrato_version, OLD.estado, NEW.estado;
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_contrato_transicion_estado
BEFORE UPDATE OF estado ON modulo7.versiones_contrato_aaef
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_versiones_contrato_transicion_estado();

COMMENT ON TRIGGER trg_versiones_contrato_transicion_estado ON
modulo7.versiones_contrato_aaef
IS 'TGR-M07-03 | RF-98 Restricción 5.1 y 5.7 | Garantiza transiciones de
estado unidireccionales en versiones de contrato AAEF:
PENDIENTE→ACTIVO→DEPRECADO→INACTIVO. Bloquea retrocesos, saltos y
reactivaciones.';

```

TGR-M07-04:

trg_versiones_contrato_limit e_activas

Descripción	Impide que existan más de 2 versiones de contrato AAEF simultáneamente en estado ACTIVO o DEPRECADO. Antes de activar una nueva versión (cambio a ACTIVO o DEPRECADO), cuenta cuántas versiones ya están en esos estados. Si hay 2 o más, bloquea la transición.
Aplicado a	RF-98
Restricción origen RF	RF-98, Restricción 5.3 — <i>"El sistema solo puede tener como máximo 2 versiones simultáneamente en estado ACTIVO o DEPRECADO."</i> RF-98, Restricción 5.6 — <i>"El sistema garantiza en todo momento que no existen más de dos versiones simultáneamente en estado ACTIVO o DEPRECADO. Esta garantía se implementa mediante restricciones transaccionales."</i>
Justificación	Esta es una restricción transaccional explícitamente mencionada en el requerimiento. Si dos procesos del backend intentan activar versiones en paralelo, solo la BD puede garantizar la consistencia mediante su mecanismo de bloqueo de transacciones.

```
CREATE OR REPLACE FUNCTION
modulo7.fn_versiones_contrato_limite_activas()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    -- Solo aplica cuando el nuevo estado es ACTIVO o DEPRECADO
    IF NEW.estado::TEXT NOT IN ('ACTIVO', 'DEPRECADO') THEN
```



```

        RETURN NEW;
    END IF;

    -- Si el estado anterior ya era ACTIVO o DEPRECADO, no suma al
    conteo
    IF OLD.estado::TEXT IN ('ACTIVO', 'DEPRECADO') THEN
        RETURN NEW;
    END IF;

    -- Contar versiones actualmente en ACTIVO o DEPRECADO (excluyendo la
    fila actual)
    SELECT COUNT(*)
    INTO v_count
    FROM modulo7.versiones_contrato_aaef
    WHERE estado::TEXT IN ('ACTIVO', 'DEPRECADO')
        AND id_version_contrato_aaef <> NEW.id_version_contrato_aaef;

    IF v_count >= 2 THEN
        RAISE EXCEPTION
            '[TGR-M07-04] Violación RF-98 Restricción 5.3: Límite de
            versiones simultáneas excedido. Ya existen % versiones en estado ACTIVO
            o DEPRECADO. Máximo permitido: 2. Versión bloqueada: %.',
            v_count,
            NEW.contrato_version;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_contrato_limite_activas
BEFORE UPDATE OF estado ON modulo7.versiones_contrato_aaef
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_versiones_contrato_limite_activas();

COMMENT ON TRIGGER trg_versiones_contrato_limite_activas ON
modulo7.versiones_contrato_aaef
IS 'TGR-M07-04 | RF-98 Restricción 5.3 y 5.6 | Impide que existan más de
2 versiones de contrato AAEF en estado ACTIVO o DEPRECADO de forma
simultánea.';

```

TGR-M07-05:

trg_versiones_contrato_inmutable_activo

Descripción	Impide modificar los campos estructurales de una versión de contrato que ya se encuentre en estado ACTIVO o DEPRECADO. Detecta intentos de modificar contrato_version, fecha_comienzo o descripcion en versiones con estado ACTIVO o DEPRECADO. Si se detecta, lanza excepción. El cambio de estado en sí está controlado por TGR-M07-03.
Aplicado a	RF-98
Restricción origen RF	RF-98, Restricción 5.12 — <i>"No se permite modificar estructura, mapeo ni comportamiento de una versión ya ACTIVA. Cualquier cambio requiere una nueva versión."</i> RF-98, RNF-98.11 — <i>"Los contratos en estado ACTIVO o DEPRECADO deben ser inmutables."</i>
Justificación	La inmutabilidad de contratos activos es una garantía de integridad que no puede depender únicamente del control de acceso del backend. Debe existir como restricción en la capa de datos.

```
CREATE OR REPLACE FUNCTION
modulo7.fn_versiones_contrato_inmutable_activo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Solo aplica si el estado actual es ACTIVO o DEPRECADO
    IF OLD.estado::TEXT NOT IN ('ACTIVO', 'DEPRECADO') THEN
        RETURN NEW;
```

```

END IF;

-- Verificar si se intenta modificar campos estructurales (distinto
al estado)
IF NEW.contrato_version <> OLD.contrato_version
OR NEW.fecha_comienzo <> OLD.fecha_comienzo
OR (NEW.descripcion IS DISTINCT FROM OLD.descripcion)
THEN
    RAISE EXCEPTION
        '[TGR-M07-05] Violación RF-98 Restricción 5.12: No se
permite modificar campos estructurales de un contrato en estado %.
Versión: %. Crear una nueva versión para realizar cambios.',
        OLD.estado,
        OLD.contrato_version;
END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_versiones_contrato_inmutable_activo
BEFORE UPDATE ON modulo7.versiones_contrato_aaef
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_versiones_contrato_inmutable_activo();

COMMENT ON TRIGGER trg_versiones_contrato_inmutable_activo ON
modulo7.versiones_contrato_aaef
IS 'TGR-M07-05 | RF-98 Restricción 5.12 y RNF-98.11 | Protege la
inmutabilidad de contratos AAEF en estado ACTIVO o DEPRECADO. Bloquea
cambios a campos estructurales; solo permite transición de estado
controlada por TGR-M07-03.';

```

TGR-M07-06:

trg_cliente_externo_estado_r evocado_irreversible

Descripción	Impide que un cliente externo con estado REVOCADO sea modificado a cualquier otro estado, garantizando la irreversibilidad de la revocación.
-------------	---

	Al detectar un UPDATE sobre el campo estado de clientes_externos donde el estado anterior es REVOCADO , lanza excepción. También valida que desde SUSPENDIDO solo se pueda ir a ACTIVO .
Aplicado a	RF-101
Restricción origen RF	RF-101, Restricción 6 — <i>"REVOCADO es irreversible. SUSPENDIDO solo puede volver a ACTIVO."</i> RF-101, Proceso Fase 4 — <i>"El sistema cambia estado → REVOCADO (irreversible)."</i>
Justificación	La irreversibilidad de la revocación es una regla de negocio de seguridad crítica. Un error en el backend o una acción directa en BD podría reactivar un cliente que fue revocado por razones de seguridad. La BD debe ser la última barrera.

```

CREATE OR REPLACE FUNCTION modulo7.fn_cliente_externo_estado_valido()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Sin cambio de estado: permitir
    IF OLD.estado IS NOT DISTINCT FROM NEW.estado THEN
        RETURN NEW;
    END IF;

    -- REVOCADO es irreversible: no se puede cambiar desde REVOCADO
    IF OLD.estado::TEXT = 'REVOCADO' THEN
        RAISE EXCEPTION
            '[TGR-M07-06] Violación RF-101 Restricción 6: El estado
REVOCADO es irreversible. Cliente: %, Código: %. No se puede cambiar a
estado: %.',
            OLD.nombre,
            OLD.codigo,
            NEW.estado;
    END IF;

    -- Desde SUSPENDIDO solo se puede ir a ACTIVO
    IF OLD.estado::TEXT = 'SUSPENDIDO' AND NEW.estado::TEXT NOT IN
('ACTIVO', 'REVOCADO') THEN
        RAISE EXCEPTION

```

```

        '[TGR-M07-06] Violación RF-101 Restricción 6: Desde estado
SUSPENDIDO solo se permite transición a ACTIVO o REVOCADO. Cliente: %,
Transición intentada: SUSPENDIDO → %.',
        OLD.nombre,
        NEW.estado;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_cliente_externo_estado_valido
BEFORE UPDATE OF estado ON modulo7.clientes_externos
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_cliente_externo_estado_valido();

COMMENT ON TRIGGER trg_cliente_externo_estado_valido ON
modulo7.clientes_externos
IS 'TGR-M07-06 | RF-101 Restricción 6 | Garantiza que el estado REVOCADO
de clientes externos sea irreversible y que desde SUSPENDIDO solo se
permita transición a ACTIVO o REVOCADO.';

```

TGR-M07-07:

trg_identificadores_apis_un_activo_por_cliente

Descripción	Garantiza que un cliente externo tenga como máximo una API Key en estado ACTIVO en un momento dado. Antes de insertar o activar una nueva API Key para un cliente, verifica que no exista ya una activa para ese mismo cliente. Si existe, lanza excepción.
Aplicado a	RF-101
Restricción origen RF	RF-101, Proceso Fase 2 (Renovación) — "Se ejecuta transacción atómica: genera nueva API Key, invalida la anterior." RF-101, CA-5 — "Dado un cliente

	<i>existente, cuando se renueva su API Key, entonces la clave anterior debe invalidarse inmediatamente y la nueva debe activarse sin solapamiento."</i>
Justificación	El solapamiento de API Keys activas para el mismo cliente es un riesgo de seguridad. Si el backend falla a mitad de una renovación, podrían quedar dos claves activas simultáneamente. La BD debe garantizar que esto nunca ocurra.

```

CREATE OR REPLACE FUNCTION modulo7.fn_identificadores_apis_un_activo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INT;
BEGIN
    -- Solo aplica si el nuevo estado es ACTIVO
    IF NEW.estado::TEXT <> 'ACTIVO' THEN
        RETURN NEW;
    END IF;

    -- Para UPDATE: si el estado anterior ya era ACTIVO para la misma
    fila, permitir
    IF TG_OP = 'UPDATE' AND OLD.estado::TEXT = 'ACTIVO' THEN
        RETURN NEW;
    END IF;

    -- Verificar que no exista otra API Key activa para el mismo cliente
    SELECT COUNT(*)
    INTO v_count
    FROM modulo7.identificadores_apis
    WHERE id_cliente_externo = NEW.id_cliente_externo
        AND estado::TEXT = 'ACTIVO'
        AND id_identificador_api <> COALESCE(NEW.id_identificador_api,
-1);

    IF v_count > 0 THEN
        RAISE EXCEPTION
            '[TGR-M07-07] Violación RF-101 CA-5: Ya existe una API Key
en estado ACTIVO para el cliente con id %. No puede existir más de una
credencial activa simultáneamente. Invalidar la anterior antes de
activar la nueva.',
            NEW.id_cliente_externo;

```

```

END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_identificadores_apis_un_activo
BEFORE INSERT OR UPDATE OF estado ON modulo7.identificadores_apis
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_identificadores_apis_un_activo();

COMMENT ON TRIGGER trg_identificadores_apis_un_activo ON
modulo7.identificadores_apis
IS 'TGR-M07-07 | RF-101 CA-5 y Proceso Fase 2 | Garantiza que un cliente
externo no tenga más de una API Key en estado ACTIVO simultáneamente,
previniendo solapamiento en procesos de renovación.';

```

TGR-M07-08:

trg_identificadores_apis_fecha_expiracion_valida

Descripción	Valida que la fecha de expiración de una API Key sea estrictamente posterior a la fecha de emisión al momento de su creación. Al insertar o actualizar una API Key, verifica que fecha_expiracion sea posterior a fecha_emision. Si no, lanza excepción.
Aplicado a	RF-101
Restricción origen RF	RF-101, Restricción 9 — "Toda API Key debe tener fecha de vencimiento. Una clave vencida es inválida automáticamente." RF-101, Entradas — "fecha_vencimiento: Fecha de expiración de la API Key (por defecto 1 año)."

Justificación	Es una validación de integridad de datos básica que debe existir en la BD para prevenir registros incoherentes independientemente de quién inserte el dato.
----------------------	---

```

CREATE OR REPLACE FUNCTION
modulo7.fn_identificadores_apis_fecha_valida()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_expiracion IS NULL THEN
        RAISE EXCEPTION
            '[TGR-M07-08] Violación RF-101 Restricción 9: La fecha de
            expiración es obligatoria para toda API Key. Cliente id: %.',
            NEW.id_cliente_externo;
    END IF;

    IF NEW.fecha_expiracion <= NEW.fecha_emision THEN
        RAISE EXCEPTION
            '[TGR-M07-08] Violación RF-101 Restricción 9: La fecha de
            expiración (%) debe ser estrictamente posterior a la fecha de emisión
            (%). Cliente id: %.',
            NEW.fecha_expiracion,
            NEW.fecha_emision,
            NEW.id_cliente_externo;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_identificadores_apis_fecha_valida
BEFORE INSERT OR UPDATE ON modulo7.identificadores_apis
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_identificadores_apis_fecha_valida();

COMMENT ON TRIGGER trg_identificadores_apis_fecha_valida ON
modulo7.identificadores_apis
IS 'TGR-M07-08 | RF-101 Restricción 9 | Garantiza que la fecha de
expiración de cada API Key sea posterior a la fecha de emisión y que no
sea nula.';

```


TGR-M07-09:

trg_mapeos_anexos_hash_obligatorio

Descripción	Valida que al insertar un nuevo anexo de mapeo AAEF, el campo <code>canonical_sha256</code> esté presente y tenga el formato correcto de hash SHA-256 (64 caracteres hexadecimales). Al insertar un registro en <code>mapeos_anexos_aaef</code>, valida que <code>canonical_sha256</code> tenga exactamente 64 caracteres hexadecimales en minúscula.
Aplicado a	RF-AE02
Restricción origen RF	RF-AE02, Restricción 5.6 — <i>"El hash_documento_anexo se calcula sobre el documento formal en su formato canónico. Si el hash recalculado no coincide con el almacenado en RF-98, el sistema genera alerta CRÍTICA y bloquea el contrato."</i> RF-AE02, CA-20 — <i>"Integridad del documento aprobado: el hash SHA-256 recalculado... coincide con hash_documento_anexo almacenado en RF-98."</i>
Justificación	El hash del documento formal es el mecanismo de verificación de integridad del Anexo. Un registro con hash nulo o malformado hace imposible la verificación posterior. La BD debe rechazar estos registros como primera línea de defensa.

```
CREATE OR REPLACE FUNCTION modulo7.fn_mapeos_anexos_hash_obligatorio()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.canonical_sha256 IS NULL OR TRIM(NEW.canonical_sha256) = ''
    THEN
```

```

        RAISE EXCEPTION
            '[TGR-M07-09] Violación RF-AE02 Restricción 5.6: El hash
            SHA-256 del documento canónico es obligatorio para registrar un anexo de
            mapeo. Código anexo: %.',
            NEW.codigo_anexo;
    END IF;

    IF LENGTH(NEW.canonical_sha256) <> 64 THEN
        RAISE EXCEPTION
            '[TGR-M07-09] Violación RF-AE02 Restricción 5.6: El hash
            SHA-256 del anexo debe tener exactamente 64 caracteres. Longitud
            recibida: %. Código anexo: %.',
            LENGTH(NEW.canonical_sha256),
            NEW.codigo_anexo;
    END IF;

    IF NEW.canonical_sha256 !~ '^[a-f0-9]{64}$' THEN
        RAISE EXCEPTION
            '[TGR-M07-09] Violación RF-AE02 Restricción 5.6: El hash
            SHA-256 del anexo contiene caracteres inválidos. Solo se admiten
            caracteres hexadecimales en minúscula. Código anexo: %.',
            NEW.codigo_anexo;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_mapeos_anexos_hash_obligatorio
BEFORE INSERT ON modulo7.mapeos_anexos_aaef
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_mapeos_anexos_hash_obligatorio();

COMMENT ON TRIGGER trg_mapeos_anexos_hash_obligatorio ON
modulo7.mapeos_anexos_aaef
IS 'TGR-M07-09 | RF-AE02 Restricción 5.6 y CA-20 | Garantiza que todo
anexo de mapeo AAEF registrado tenga un hash SHA-256 válido del
documento canónico formal.';

```

TGR-M07-10:

trg_mapeos_anexos_aprobacion_interna_agrofusion

Descripción	Impide que se registre una fecha de aprobación de Agrofusión (fecha_aprobacion_agrofusion) sin que exista previamente una fecha de aprobación interna (fecha_aprobacion_interna). Al insertar o actualizar un registro en mapeos_anexos_aaef, si fecha_aprobacion_agrofusion tiene valor pero fecha_aprobacion_interna es nula, lanza excepción. Adicionalmente valida que la aprobación de Agrofusión no sea anterior a la aprobación interna.
Aplicado a	RF-AE02
Restricción origen RF	RF-AE02, Restricción 5.9 — "La aprobación interna del Equipo Contable/Financiero es una precondition para que el Administrador cambie el estado a <i>EN_REVISION</i> y lo prepare para envío externo." RF-AE02, CA-31 — "La aprobación interna del Equipo Contable/Financiero es registrada en el sistema (RF-102) antes de que el Administrador cambie el estado del Anexo a <i>EN_REVISION</i> ."
Justificación	El orden de aprobaciones es una restricción de proceso crítica. Si se registra la aprobación de Agrofusión sin aprobación interna previa, el proceso quedó mal ejecutado. La BD puede validar esta secuencia como integridad de datos.

```
CREATE OR REPLACE FUNCTION modulo7.fn_mapeos_anexos_orden_aprobaciones()  
RETURNS TRIGGER
```

```

LANGUAGE plpgsql
AS $$
BEGIN
    -- Si se intenta registrar aprobación de Agrofusión sin aprobación
    interna previa
    IF NEW.fecha_aprobacion_agrofusion IS NOT NULL
        AND NEW.fecha_aprobacion_interna IS NULL THEN
        RAISE EXCEPTION
            '[TGR-M07-10] Violación RF-AE02 Restricción 5.9: No se puede
            registrar la aprobación de Agrofusión sin que exista aprobación interna
            previa del equipo contable. Código anexo: %.',
            NEW.codigo_anexo;
    END IF;

    -- Validar que la aprobación de Agrofusión no sea anterior a la
    aprobación interna
    IF NEW.fecha_aprobacion_agrofusion IS NOT NULL
        AND NEW.fecha_aprobacion_interna IS NOT NULL
        AND NEW.fecha_aprobacion_agrofusion <
        NEW.fecha_aprobacion_interna THEN
        RAISE EXCEPTION
            '[TGR-M07-10] Violación RF-AE02 Restricción 5.9: La fecha de
            aprobación de Agrofusión (%) no puede ser anterior a la fecha de
            aprobación interna (%). Código anexo: %.',
            NEW.fecha_aprobacion_agrofusion,
            NEW.fecha_aprobacion_interna,
            NEW.codigo_anexo;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_mapeos_anexos_orden_aprobaciones
BEFORE INSERT OR UPDATE ON modulo7.mapeos_anexos_aaef
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_mapeos_anexos_orden_aprobaciones();

COMMENT ON TRIGGER trg_mapeos_anexos_orden_aprobaciones ON
modulo7.mapeos_anexos_aaef
IS 'TGR-M07-10 | RF-AE02 Restricción 5.9 y CA-31 | Garantiza que la
aprobación de Agrofusión solo pueda registrarse después de la aprobación
interna del equipo contable, y que las fechas sean cronológicamente
coherentes.';

```

TGR-M07-11:

trg_metadatos_respuesta_aaef_hash_obligatorio

Descripción	Valida que el hash SHA-256 (sha256_sobre) del envelope AAEF esté presente y sea formalmente válido al registrar una respuesta generada. Al insertar en metadatos_respuesta_aaef, valida que sha256_sobre tenga 64 caracteres hexadecimales válidos.
Aplicado a	RF-97, RF-AE01
Restricción origen RF	RF-97, Fase 9 — <i>"Hash SHA-256 calculado sobre el contenido canónico excluyendo el campo generatedAt. El hash es determinístico para el mismo período y datos."</i> RF-AE01, RNF-AE01.5 — <i>"El sistema debe garantizar cálculo correcto del hash de verificación y consistencia entre contenido y hash."</i>
Justificación	El hash del envelope es el mecanismo de verificación de integridad que Agrofusión usa para validar el contenido recibido. Un envelope registrado sin hash hace imposible la auditoría posterior. La BD debe rechazar registros sin este campo crítico.

```
CREATE OR REPLACE FUNCTION
modulo7.fn_metadatos_respuesta_hash_obligatorio()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.sha256_sobre IS NULL OR TRIM(NEW.sha256_sobre) = '' THEN
        RAISE EXCEPTION
            '[TGR-M07-11] Violación RF-97/RF-AE01: El hash SHA-256 del
envelope AAEF es obligatorio. Solicitud id: %.',
```

```

        NEW.id_integracion_solicitud;
    END IF;

    IF LENGTH(NEW.sha256_sobre) <> 64 THEN
        RAISE EXCEPTION
            '[TGR-M07-11] Violación RF-97/RF-AE01: El hash SHA-256 del
envelope debe tener exactamente 64 caracteres. Longitud recibida: %.
Solicitud id: %.',
            LENGTH(NEW.sha256_sobre),
            NEW.id_integracion_solicitud;
    END IF;

    IF NEW.sha256_sobre !~ '^[a-f0-9]{64}$' THEN
        RAISE EXCEPTION
            '[TGR-M07-11] Violación RF-97/RF-AE01: El hash SHA-256 del
envelope contiene caracteres inválidos. Solo se admiten hexadecimales en
minúscula. Solicitud id: %.',
            NEW.id_integracion_solicitud;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_metadatos_respuesta_hash_obligatorio
BEFORE INSERT ON modulo7.metadatos_respuesta_aaef
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_metadatos_respuesta_hash_obligatorio();

COMMENT ON TRIGGER trg_metadatos_respuesta_hash_obligatorio ON
modulo7.metadatos_respuesta_aaef
IS 'TGR-M07-11 | RF-97 Fase 9 y RF-AE01 RNF-AE01.5 | Valida que el hash
SHA-256 del envelope AAEF esté presente y sea formalmente válido al
registrar una respuesta generada.';

```

TGR-M07-12:

trg_notificaciones_webhook_p payload_hash_obligatorio

Descripción	Valida que el hash SHA-256 del payload del
-------------	--

	webhook (payload_sha256) esté presente y sea formalmente válido al registrar un intento de envío. Al insertar en notificaciones_weebhook, valida que payload_sha256 tenga formato válido de 64 caracteres hexadecimales.
Aplicado a	RF-99
Restricción origen RF	RF-99, Restricción — <i>"Firma obligatoria. Cada Webhook debe incluir X-Webhook-Signature. La firma debe generarse con HMAC-SHA256."</i> RF-99, Restricción — <i>"Firma HMAC — definición exacta: La firma resultante se expresa en formato hexadecimal minúscula de 64 caracteres."</i>
Justificación	El hash del payload es evidencia de integridad del webhook enviado, necesaria para auditoría y verificación posterior. Un registro de webhook sin hash no puede ser verificado ni auditado correctamente.

```

CREATE OR REPLACE FUNCTION modulo7.fn_webhook_payload_hash_obligatorio()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.payload_sha256 IS NULL OR TRIM(NEW.payload_sha256) = '' THEN
        RAISE EXCEPTION
            '[TGR-M07-12] Violación RF-99: El hash SHA-256 del payload
del webhook es obligatorio para garantizar trazabilidad e integridad.
Solicitud id: %.',
            NEW.id_integracion_solicitudes;
    END IF;

    IF LENGTH(NEW.payload_sha256) <> 64 THEN
        RAISE EXCEPTION
            '[TGR-M07-12] Violación RF-99: El hash SHA-256 del payload
webhook debe tener exactamente 64 caracteres. Longitud recibida: %.
Solicitud id: %.',
            LENGTH(NEW.payload_sha256),

```

```

        NEW.id_integracion_solicitudes;
    END IF;

    IF NEW.payload_sha256 !~ '^[a-f0-9]{64}$' THEN
        RAISE EXCEPTION
            '[TGR-M07-12] Violación RF-99: El hash SHA-256 del payload
webhook contiene caracteres inválidos. Solo se admiten hexadecimales en
minúscula. Solicitud id: %.',
            NEW.id_integracion_solicitudes;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_webhook_payload_hash_obligatorio
BEFORE INSERT ON modulo7.notificaciones_weebhook
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_webhook_payload_hash_obligatorio();

COMMENT ON TRIGGER trg_webhook_payload_hash_obligatorio ON
modulo7.notificaciones_weebhook
IS 'TGR-M07-12 | RF-99 Restricción de firma obligatoria | Valida que el
hash SHA-256 del payload de cada webhook registrado sea válido y esté
presente, garantizando trazabilidad e integridad.';

```

TGR-M07-13:

trg_integraciones_solicitudes_fecha_fecha_coherentes

Descripción	Valida que la fecha de finalización de una solicitud de integración sea posterior o igual a la fecha de inicio, evitando registros con duraciones incoherentes. Al insertar o actualizar en <code>integraciones_solicitudes</code>, verifica que <code>fecha_finalizacion</code> <code>>=</code> <code>fecha_comienzo</code>. Si no, lanza excepción.
-------------	--

Aplicado a	RF-95, RF-102
Restricción origen RF	RF-95, RNF-95.2 — <i>"Tiempo total de respuesta incluyendo RF-97: máximo 5 segundos."</i> RF-102, Restricción 4.7 — <i>"Normalización temporal: todos los timestamps deben estar en formato UTC."</i> RF-102 — Registro completo obligatorio; datos inconsistentes no deben persistirse.
Justificación	La coherencia temporal de los registros de solicitud es fundamental para la trazabilidad y para el cálculo correcto de la duración. Registros con fecha_finalizacion anterior a fecha_comienzo producirían métricas de rendimiento erróneas y auditorías inválidas.

```

CREATE OR REPLACE FUNCTION modulo7.fn_integraciones_fechas_coherentes()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_finalizacion IS NOT NULL
        AND NEW.fecha_comienzo IS NOT NULL
        AND NEW.fecha_finalizacion < NEW.fecha_comienzo THEN
        RAISE EXCEPTION
            '[TGR-M07-13] Violación RF-95/RF-102: La fecha de
finalización (%) no puede ser anterior a la fecha de inicio (%) para la
solicitud de integración id: %.',
            NEW.fecha_finalizacion,
            NEW.fecha_comienzo,
            NEW.id_integracion_solicitud;
    END IF;

    RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_integraciones_fechas_coherentes
BEFORE INSERT OR UPDATE ON modulo7.integraciones_solicitudes
FOR EACH ROW
EXECUTE FUNCTION modulo7.fn_integraciones_fechas_coherentes();

COMMENT ON TRIGGER trg_integraciones_fechas_coherentes ON
modulo7.integraciones_solicitudes

```

IS 'TGR-M07-13 | RF-95 RNF-95.2 y RF-102 Restricción 4.7 | Garantiza coherencia temporal entre fecha de inicio y finalización en registros de solicitudes de integración.';